

```
In [ ]: import marimo as mo
```

TP1.1 — Gerador de Horário Escolar

Objetivo e abordagem

O objetivo é construir um horário semanal a partir de ficheiros CSV, respeitando as restrições obrigatórias **R1–R8**, minimizando os **buracos dos professores (O1)** e permitindo **reparação incremental (R9)** quando os recursos mudam.

A solução foi modelada como um problema de satisfação e otimização com **OR-Tools CP-SAT**. Esta escolha segue a abordagem de alocação estudada na disciplina: as decisões são representadas por variáveis booleanas e as condições do horário por restrições. O CP-SAT permite ainda definir funções objetivo e reutilizar uma solução anterior através de *hints*, o que é útil em O1 e R9.

Os dados concretos de turmas, disciplinas, professores e salas **não são escritos no modelo**. São lidos de `dados/`, `dados_v2/` e, no teste adicional de reparação, de `dados_reparo/`. Para a leitura é usado o módulo `csv` da biblioteca standard: os ficheiros são pequenos e têm uma estrutura simples, pelo que não é necessário acrescentar a dependência `pandas`, e a transformação dos campos fica explícita no código.

O horário extraído é representado como uma **lista de dicionários**, uma estrutura simples que facilita a inspeção dos resultados, a validação independente e a comparação das alocações entre H0 e H1.

Requisitos considerados

- **R1:** uma turma não pode ter duas aulas simultâneas.
- **R2:** cada turma cumpre exatamente a carga semanal de cada disciplina.
- **R3:** existe no máximo uma ocorrência da mesma disciplina por dia.
- **R4:** disciplinas de duplo período são dadas em blocos de dois períodos consecutivos.
- **R5:** um professor não pode dar duas aulas simultaneamente.
- **R6:** os professores só podem ser colocados em períodos disponíveis.
- **R7:** a sala tem de ser adequada e não pode receber duas aulas simultaneamente.
- **R8:** os dados são lidos dos CSV e a solução não depende dos valores concretos do exemplo.
- **O1:** minimizar o número total de períodos livres entre a primeira e a última aula diária de cada professor.
- **R9:** adaptar um horário existente a pequenas alterações, preservando o maior número possível de alocações antigas.

As decisões de implementação que não vêm diretamente do enunciado são identificadas ao longo do relatório e resumidas na matriz de rastreabilidade final.

```
In [ ]: # [LLM] Implementação inicial do carregamento parametrizado dos CSV e da
# expansão das salas individuais, gerada com auxílio de IA após discussão
# e aprovação de R8, D1 e D4; a integrar, adaptar e validar pelo grupo.

from pathlib import Path
import csv

from ortools.sat.python import cp_model

# Domínio temporal fixado pelo enunciado, não pelos dados de exemplo.
DIAS = ("Seg", "Ter", "Qua", "Qui", "Sex")
PERIODOS = (1, 2, 3, 4, 5)

def ler_csv(caminho):
    """Lê um CSV e devolve as linhas como dicionários."""
    with open(caminho, newline="", encoding="utf-8-sig") as ficheiro:
        return list(csv.DictReader(ficheiro))

def carregar_dados(pasta):
    """
    Carrega os quatro ficheiros CSV da pasta indicada.

    A única validação estrutural explicitamente decidida até agora (D4)
    é rejeitar carga semanal ímpar numa disciplina de duplo período.
    """
    pasta = Path(pasta)

    linhas_turmas = ler_csv(pasta / "turmas.csv")
    linhas_disciplinas = ler_csv(pasta / "disciplinas.csv")
    linhas_salas = ler_csv(pasta / "salas.csv")
    linhas_indisponibilidade = ler_csv(
        pasta / "disponibilidade_excecoes.csv"
    )

    turmas = [
        linha["turma"].strip()
        for linha in linhas_turmas
    ]

    disciplinas = []

    for linha in linhas_disciplinas:
        carga = int(linha["carga_semanal"])
        duplo = linha["duplo_perodo"].strip().lower()

        # D4 já decidida pelo grupo.
        if duplo == "sim" and carga % 2 != 0:
            raise ValueError(
                f"A disciplina {linha['disciplina']!r} está marcada "
```

```

        f"como duplo período mas tem carga semanal ímpar ({carga})."
    )

    disciplinas.append(
        {
            "disciplina": linha["disciplina"].strip(),
            "professor": linha["professor"].strip(),
            "carga_semanal": carga,
            "duplo_periodo": duplo,
            "sala_especial": linha["sala_especial"].strip(),
        }
    )

# D1: transformar quantidade=q em q salas concretas equivalentes.
salas = []

for linha in linhas_salas:
    nome_base = linha["sala"].strip()
    tipo = linha["tipo"].strip()
    quantidade = int(linha["quantidade"])

    for numero in range(1, quantidade + 1):
        salas.append(
            {
                "id": f"{nome_base} #{numero}",
                "base": nome_base,
                "tipo": tipo,
            }
        )

indisponibilidades = {
    (
        linha["professor"].strip(),
        linha["dia"].strip(),
        int(linha["periodo"]),
    )
    for linha in linhas_indisponibilidade
}

return {
    "turmas": turmas,
    "disciplinas": disciplinas,
    "salas": salas,
    "indisponibilidades": indisponibilidades,
    "dias": DIAS,
    "periodos": PERIODOS,
}

```

Modelação de R1–R8

Dados e decisões de representação

Sejam **T** as turmas, **C** as disciplinas, **D** os dias, **P** os períodos e **S** as salas concretas.

A variável principal é

$$x_{t,c,d,p,s} \in \{0,1\},$$

com $x = 1$ se a turma t tiver a disciplina c , no dia d , período p , na sala s .

D1 — salas concretas. Uma linha de `salas.csv` com `quantidade = q` é expandida em q salas equivalentes, por exemplo `Sala Normal #1`, ..., `Sala Normal #q`. Os identificadores são internos; servem para aplicar diretamente a capacidade de R7 e para comparar salas em R9.

D2 — variável principal. Foi escolhida a matriz binária x por ser próxima da formulação de alocação usada na disciplina. A variável auxiliar $y[t,c,d,p]$ indica apenas a ocupação temporal da turma/disciplina, independentemente da sala.

D3 — blocos duplos. R4 exige dois períodos consecutivos, mas não exige que os dois períodos usem a mesma sala. Não foi acrescentada essa restrição extra.

D4 — validação estrutural. Antes de construir o modelo rejeita-se uma disciplina marcada como `duplo_periodo = sim` com carga semanal ímpar. Esta é uma decisão de qualidade dos dados e não uma nova interpretação de R8.

Restrições

- **R1:** para cada turma e slot, $\text{sum}(x) \leq 1$.
- **R2:** para cada turma/disciplina, a soma semanal de x é exatamente a carga indicada no CSV.
- **R3:** disciplinas normais têm no máximo um período por dia; disciplinas duplas têm no máximo um bloco por dia.
- **R4:** a variável binária $b[t,c,d,p]$ representa o início de um bloco duplo e liga y a exatamente dois períodos consecutivos.
- **R5:** para cada professor e slot, a soma das suas aulas é no máximo 1.
- **R6:** se $(\text{professor}, \text{dia}, \text{periodo})$ estiver nas exceções de disponibilidade, a respetiva ocupação é forçada a 0.
- **R7:** variáveis associadas a salas incompatíveis são forçadas a 0; cada sala concreta recebe no máximo uma aula por slot.
- **R8:** o modelo é construído a partir dos CSV, sem listas de turmas, disciplinas ou professores concretos no código.

```
In [ ]: # [LLM] Implementação inicial do modelo CP-SAT de R1-R7 gerada com auxílio
# de IA após discussão e aprovação das variáveis x, das restrições R1-R7
# e das decisões D1-D4; a integrar, adaptar e validar pelo grupo.
```

```
def construir_modelo(dados):
    model = cp_model.CpModel()

    turmas = dados["turmas"]
```

```

disciplinas = dados["disciplinas"]
salas = dados["salas"]
dias = dados["dias"]
periodos = dados["periodos"]
indisponibilidades = dados["indisponibilidades"]

T = range(len(turmas))
C = range(len(disciplinas))
D = range(len(dias))
P = range(len(periodos))
S = range(len(salas))

# -----
# Variável principal D2:
#
#  $x[t,c,d,p,s] = 1$  sse a turma  $t$  tem a disciplina  $c$ ,
# no dia  $d$ , período  $p$ , na sala  $s$ .
# -----

x = {}

for t in T:
    for c in C:
        for d in D:
            for p in P:
                for s in S:
                    x[t, c, d, p, s] = model.NewBoolVar(
                        f"x_{t}_{c}_{d}_{p}_{s}"
                    )

# Variável auxiliar:
#
#  $y[t,c,d,p] = 1$  sse a disciplina  $c$  da turma  $t$ 
# ocorre no dia  $d$ , período  $p$ , independentemente da sala.

y = {}

for t in T:
    for c in C:
        for d in D:
            for p in P:
                y[t, c, d, p] = model.NewBoolVar(
                    f"y_{t}_{c}_{d}_{p}"
                )

                model.Add(
                    y[t, c, d, p]
                    == sum(x[t, c, d, p, s] for s in S)
                )

# =====
# R1
# Uma turma não pode ter duas aulas simultâneas.
# =====

for t in T:

```

```

    for d in D:
        for p in P:
            model.Add(
                sum(
                    x[t, c, d, p, s]
                    for c in C
                    for s in S
                )
                <= 1
            )

# =====
# R2
# Cada disciplina cumpre exatamente a carga semanal
# para cada turma.
# =====

for t in T:
    for c in C:
        carga = disciplinas[c]["carga_semanal"]

        model.Add(
            sum(
                x[t, c, d, p, s]
                for d in D
                for p in P
                for s in S
            )
            == carga
        )

# =====
# R3 + R4
#
# Disciplinas normais:
# no máximo um período por dia.
#
# Disciplinas de duplo período:
# são representadas por inícios de blocos b;
# existe no máximo um bloco por dia;
# cada bloco ocupa exatamente dois períodos consecutivos.
# =====

b = {}

for t in T:
    for c in C:
        duplo = disciplinas[c]["duplo_periodo"] == "sim"

        for d in D:

            if not duplo:
                # R3 para disciplinas normais.
                model.Add(
                    sum(y[t, c, d, p] for p in P) <= 1
                )

```

```

else:
    # Um bloco só pode começar até ao penúltimo período.
    inicios = range(len(periodos) - 1)

    for p in inicios:
        b[t, c, d, p] = model.NewBoolVar(
            f"b_{t}_{c}_{d}_{p}"
        )

    # R3 para disciplinas de duplo período:
    # no máximo uma ocorrência/bloco por dia.
    model.Add(
        sum(b[t, c, d, p] for p in inicios) <= 1
    )

    # R4:
    # cada período ocupado pertence a um bloco
    # que começa nesse período ou no anterior.
    for p in P:
        blocos_que_cobrem = []

        if p > 0:
            blocos_que_cobrem.append(
                b[t, c, d, p - 1]
            )

        if p < len(periodos) - 1:
            blocos_que_cobrem.append(
                b[t, c, d, p]
            )

        model.Add(
            y[t, c, d, p]
            == sum(blocos_que_cobrem)
        )

# =====
# R5
# Um professor não pode dar duas aulas simultaneamente.
# =====

professores = sorted(
    {
        disciplina["professor"]
        for disciplina in disciplinas
    }
)

for professor in professores:
    disciplinas_professor = [
        c
        for c in C
        if disciplinas[c]["professor"] == professor
    ]

```

```

    for d in D:
        for p in P:
            model.Add(
                sum(
                    x[t, c, d, p, s]
                    for t in T
                    for c in disciplinas_professor
                    for s in S
                )
                <= 1
            )

# =====
# R6
# Um professor só pode dar aulas em períodos disponíveis.
# =====

for t in T:
    for c in C:
        professor = disciplinas[c]["professor"]

        for d in D:
            dia = dias[d]

            for p in P:
                periodo = periodos[p]

                if (
                    professor,
                    dia,
                    periodo,
                ) in indisponibilidades:
                    model.Add(
                        y[t, c, d, p] == 0
                    )

# =====
# R7a
# A sala utilizada tem de ser adequada à disciplina.
# =====

for t in T:
    for c in C:
        sala_especial = disciplinas[c]["sala_especial"]

        for d in D:
            for p in P:
                for s in S:
                    sala = salas[s]

                    if sala_especial:
                        admissivel = (
                            sala["base"] == sala_especial
                        )
                    else:
                        admissivel = (

```



```

        sala["tipo"] == "normal"
    )

    if not admissivel:
        model.Add(
            x[t, c, d, p, s] == 0
        )

# =====
# R7b
# Cada sala concreta só pode receber uma aula simultaneamente.
# =====

for s in S:
    for d in D:
        for p in P:
            model.Add(
                sum(
                    x[t, c, d, p, s]
                    for t in T
                    for c in C
                )
                <= 1
            )

return model, x, y, b

```

Objetivo O1 — minimizar buracos

A definição de **buraco** vem do enunciado: é cada período livre entre a primeira e a última aula diária de um professor.

D5 — decisão de modelação. O objetivo é representado por quatro famílias de variáveis booleanas:

- `ocupado[f, d, p]` : o professor tem uma aula nesse slot;
- `tem_antes[f, d, p]` : existe pelo menos uma aula anterior nesse dia;
- `tem_depois[f, d, p]` : existe pelo menos uma aula posterior nesse dia;
- `buraco[f, d, p]` : existe aula antes e depois, mas o próprio período está livre.

As restrições implementam equivalências, e não apenas implicações, para impedir que o solver escolha valores artificiais para diminuir o objetivo. Em particular,

$$\text{buraco} \iff \text{tem_antes} \wedge \text{tem_depois} \wedge \neg \text{ocupado}.$$

A função objetivo é

$$\min \sum_{f,d,p} \text{buraco}_{f,d,p}.$$

R1–R8 permanecem inalterados; O1 é acrescentado ao modelo base.

```

In [ ]: # [LLM] Implementação de 01 em CP-SAT, gerada com auxílio de IA
# após discussão e aprovação da formalização.
#
# R1-R8 permanecem inalterados. Esta função recebe o modelo base
# e acrescenta apenas as variáveis, restrições e objetivo de 01.

def adicionar_objetivo_buracos(model, dados, x):
    turmas = dados["turmas"]
    disciplinas = dados["disciplinas"]
    salas = dados["salas"]
    dias = dados["dias"]
    periodos = dados["periodos"]

    T = range(len(turmas))
    C = range(len(disciplinas))
    D = range(len(dias))
    P = range(len(periodos))
    S = range(len(salas))

    professores = sorted(
        {
            disciplina["professor"]
            for disciplina in disciplinas
        }
    )

    ocupado = {}
    tem_antes = {}
    tem_depois = {}
    buraco = {}

    # -----
    # ocupado[f,d,p] = 1 sse o professor f dá uma aula
    # nesse dia e período.
    #
    # R5 já garante que esta soma é <= 1.
    # -----

    for f, professor in enumerate(professores):
        disciplinas_professor = [
            c
            for c in C
            if disciplinas[c]["professor"] == professor
        ]

        for d in D:
            for p in P:
                ocupado[f, d, p] = model.NewBoolVar(
                    f"ocupado_{f}_{d}_{p}"
                )

                model.Add(
                    ocupado[f, d, p]
                    == sum(
                        x[t, c, d, p, s]

```

```

        for t in T
        for c in disciplinas_professor
        for s in S
    )
)

# -----
# tem_antes[f,d,p] = 1 sse existe pelo menos uma aula
# anterior do professor nesse dia.
# -----

for f, professor in enumerate(professores):
    for d in D:
        for p in P:
            tem_antes[f, d, p] = model.NewBoolVar(
                f"tem_antes_{f}_{d}_{p}"
            )

            ocupacoes_anteriores = [
                ocupado[f, d, q]
                for q in range(p)
            ]

            if len(ocupacoes_anteriores) == 0:
                model.Add(
                    tem_antes[f, d, p] == 0
                )
            else:
                # Se existir qualquer aula anterior,
                # tem_antes é forçada a 1.
                for ocupacao in ocupacoes_anteriores:
                    model.Add(
                        tem_antes[f, d, p] >= ocupacao
                    )

                # Se não existir nenhuma aula anterior,
                # tem_antes é forçada a 0.
                model.Add(
                    tem_antes[f, d, p]
                    <= sum(ocupacoes_anteriores)
                )

# -----
# tem_depois[f,d,p] = 1 sse existe pelo menos uma aula
# posterior do professor nesse dia.
# -----

for f, professor in enumerate(professores):
    for d in D:
        for p in P:
            tem_depois[f, d, p] = model.NewBoolVar(
                f"tem_depois_{f}_{d}_{p}"
            )

            ocupacoes_posteriores = [
                ocupado[f, d, q]

```

```

        for q in range(p + 1, len(periodos))
    ]

    if len(ocupacoes_posteriores) == 0:
        model.Add(
            tem_depois[f, d, p] == 0
        )
    else:
        for ocupacao in ocupacoes_posteriores:
            model.Add(
                tem_depois[f, d, p] >= ocupacao
            )

        model.Add(
            tem_depois[f, d, p]
            <= sum(ocupacoes_posteriores)
        )

# -----
# buraco[f,d,p] = 1 sse:
#
#   tem_antes = 1
#   tem_depois = 1
#   ocupado = 0
#
# As quatro restrições implementam a equivalência completa.
# -----

for f, professor in enumerate(professores):
    for d in D:
        for p in P:
            buraco[f, d, p] = model.NewBoolVar(
                f"buraco_{f}_{d}_{p}"
            )

            model.Add(
                buraco[f, d, p]
                <= tem_antes[f, d, p]
            )

            model.Add(
                buraco[f, d, p]
                <= tem_depois[f, d, p]
            )

            model.Add(
                buraco[f, d, p]
                <= 1 - ocupado[f, d, p]
            )

            model.Add(
                buraco[f, d, p]
                >= (
                    tem_antes[f, d, p]
                    + tem_depois[f, d, p]
                    - ocupado[f, d, p]
                )
            )

```

```

        - 1
    )
)

# 01
model.Minimize(
    sum(buraco.values())
)

return {
    "professores": professores,
    "ocupado": ocupado,
    "tem_antes": tem_antes,
    "tem_depois": tem_depois,
    "buraco": buraco,
}

```

In []: *# [LLM] Função de resolução CP-SAT gerada com auxílio de IA.
A semente e o número de workers são fixados para tornar as
execuções reproduzíveis nas mesmas condições de ambiente.*

```

def resolver_modelo(modelo):
    solver = cp_model.CpSolver()
    solver.parameters.random_seed = 0
    solver.parameters.num_search_workers = 1
    estado = solver.Solve(modelo)

    return solver, estado

```

In []: *# [LLM] Implementação inicial da extração do horário gerada com auxílio
de IA; converte a instanciación das variáveis x numa estrutura Python
legível que será posteriormente usada pelo validador independente.*

```

def extrair_horario(dados, x, solver, estado):
    if estado not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
        return None

    turmas = dados["turmas"]
    disciplinas = dados["disciplinas"]
    salas = dados["salas"]
    dias = dados["dias"]
    periodos = dados["periodos"]

    horario = []

    for t, turma in enumerate(turmas):
        for c, disciplina in enumerate(disciplinas):
            for d, dia in enumerate(dias):
                for p, periodo in enumerate(periodos):
                    for s, sala in enumerate(salas):

                        if solver.Value(x[t, c, d, p, s]) == 1:
                            horario.append(
                                {
                                    "turma": turma,

```

```

        "disciplina": disciplina["disciplina"],
        "professor": disciplina["professor"],
        "dia": dia,
        "periodo": periodo,
        "sala": sala["id"],
    }
)

return horario

```

Execução inicial e validação

Primeiro resolve-se apenas o problema de satisfação R1–R8 sobre `dados/`. Esta execução permite testar a formulação e produzir um horário de referência antes de acrescentar O1. O estado e o horário obtidos são apresentados diretamente pela célula seguinte; como ainda não existe função objetivo, um estado `OPTIMAL` nesta fase não significa que os buracos tenham sido otimizados.

```

In [ ]: # [LLM] Execução inicial do modelo base e extração do horário.
dados = carregar_dados("dados")

modelo, x, y, b = construir_modelo(dados)

solver, estado = resolver_modelo(modelo)

print("Estado:", solver.StatusName(estado))

horario = extrair_horario(
    dados,
    x,
    solver,
    estado,
)

horario

```

Estado: OPTIMAL

Validação independente

Uma solução devolvida pelo solver não é aceite sem verificação adicional. O validador abaixo relê os CSV e verifica diretamente o horário extraído, sem consultar o modelo CP-SAT, as variáveis `x`, `y`, `b` ou o estado do solver.

São verificados R1–R7 e a consistência do resultado com os CSV para R8. A ausência de *hardcoding* é testada mais adiante através de um conjunto alternativo de CSV. Os resultados da validação são sempre produzidos pelas células de execução, evitando copiar manualmente estados ou contagens para o texto do relatório.

```
In [ ]: # [LLM] Validador independente de R1-R7 e da consistência com os CSV em R8.  
# Relê os ficheiros e não depende do modelo CP-SAT.
```

```
from collections import defaultdict  
  
def ler_referencia_csv_validacao(pasta):  
    pasta = Path(pasta)  
  
    def ler(nome):  
        with open(pasta / nome, newline="", encoding="utf-8-sig") as f:  
            return list(csv.DictReader(f))  
  
    turmas = {linha["turma"].strip() for linha in ler("turmas.csv")}  
    disciplinas = {  
        linha["disciplina"].strip(): {  
            "professor": linha["professor"].strip(),  
            "carga_semanal": int(linha["carga_semanal"]),  
            "duplo_perodo": linha["duplo_perodo"].strip().lower() == "sim",  
            "sala_especial": linha["sala_especial"].strip(),  
        }  
        for linha in ler("disciplinas.csv")  
    }  
  
    salas = {}  
    for linha in ler("salas.csv"):  
        base = linha["sala"].strip()  
        for numero in range(1, int(linha["quantidade"]) + 1):  
            salas[f"{base} #{numero}"] = {  
                "base": base,  
                "tipo": linha["tipo"].strip(),  
            }  
  
    indisponibilidades = {  
        (linha["professor"].strip(), linha["dia"].strip(), int(linha["periodo"]))  
        for linha in ler("disponibilidade_excecoes.csv")  
    }  
    return turmas, disciplinas, salas, indisponibilidades  
  
def validar_horario(pasta_dados, horario):  
    turmas, disciplinas, salas, indisponibilidades = (  
        ler_referencia_csv_validacao(pasta_dados)  
    )  
    erros = {f"R{i}": [] for i in range(1, 9)}  
  
    # R8 – consistência do horário com os CSV relidos.  
    for i, aula in enumerate(horario):  
        turma = aula["turma"]  
        disciplina = aula["disciplina"]  
        professor = aula["professor"]  
        sala = aula["sala"]  
        if turma not in turmas:  
            erros["R8"].append(f"Entrada {i}: turma desconhecida {turma!r}.")  
        if disciplina not in disciplinas:  
            erros["R8"].append(  
                f"Entrada {i}: disciplina desconhecida {disciplina!r}."
```

```

    )
    elif professor != disciplinas[disciplina]["professor"]:
        erros["R8"].append(
            f"Entrada {i}: {disciplina!r} aparece com {professor!r}, "
            f"mas o CSV indica {disciplinas[disciplina]['professor']!r}."
        )
    if sala not in salas:
        erros["R8"].append(f"Entrada {i}: sala desconhecida {sala!r}.")

por_turma_slot = defaultdict(list)
por_turma_disciplina = defaultdict(int)
por_turma_disciplina_dia = defaultdict(list)
por_professor_slot = defaultdict(list)
por_sala_slot = defaultdict(list)

for aula in horario:
    por_turma_slot[(aula["turma"], aula["dia"], aula["periodo"])].append(aula)
    por_turma_disciplina[(aula["turma"], aula["disciplina"])] += 1
    por_turma_disciplina_dia[
        (aula["turma"], aula["disciplina"], aula["dia"])
    ].append(aula["periodo"])
    por_professor_slot[
        (aula["professor"], aula["dia"], aula["periodo"])
    ].append(aula)
    por_sala_slot[(aula["sala"], aula["dia"], aula["periodo"])].append(aula)

# R6
if (aula["professor"], aula["dia"], aula["periodo"]) in
indisponibilidades:
    erros["R6"].append(
        f"{aula['professor']} está indisponível em {aula['dia']}, "
        f"período {aula['periodo']}, mas foi colocado em "
        f"{aula['turma']} / {aula['disciplina']}."
    )

# R7a
if aula["disciplina"] in disciplinas and aula["sala"] in salas:
    info = disciplinas[aula["disciplina"]]
    sala = salas[aula["sala"]]
    admissivel = (
        sala["base"] == info["sala_especial"]
        if info["sala_especial"]
        else sala["tipo"] == "normal"
    )
    if not admissivel:
        erros["R7"].append(
            f"{aula['turma']} / {aula['disciplina']}: "
            f"sala inadequada {aula['sala']!r}."
        )

# R1
for (turma, dia, periodo), aulas in por_turma_slot.items():
    if len(aulas) > 1:
        erros["R1"].append(
            f"{turma}: {len(aulas)} aulas simultâneas em {dia}, período
{periodo}."

```



```

    )

# R2
for turma in turmas:
    for disciplina, info in disciplinas.items():
        observada = por_turma_disciplina[(turma, disciplina)]
        if observada != info["carga_semanal"]:
            erros["R2"].append(
                f"{turma} / {disciplina}: carga observada {observada}, "
                f"esperada {info['carga_semanal']}."
            )

# R3 e R4
for (turma, disciplina, dia), periodos_dia in
por_turma_disciplina_dia.items():
    if disciplina not in disciplinas:
        continue
    info = disciplinas[disciplina]
    limite = 2 if info["duplo_periodo"] else 1
    if len(periodos_dia) > limite:
        erros["R3"].append(
            f"{turma} / {disciplina}: {len(periodos_dia)} tempos em {dia}; "
            f"máximo permitido {limite}."
        )
    if info["duplo_periodo"]:
        periodos_ordenados = sorted(periodos_dia)
        if len(periodos_ordenados) != 2:
            erros["R4"].append(
                f"{turma} / {disciplina} / {dia}: bloco com "
                f"{len(periodos_ordenados)} tempos ({periodos_ordenados}), em
vez de 2."
            )
        elif periodos_ordenados[1] != periodos_ordenados[0] + 1:
            erros["R4"].append(
                f"{turma} / {disciplina} / {dia}: períodos "
                f"{periodos_ordenados} não são consecutivos."
            )

# R5
for (professor, dia, periodo), aulas in por_professor_slot.items():
    if len(aulas) > 1:
        erros["R5"].append(
            f"{professor}: {len(aulas)} aulas simultâneas em {dia}, período
{periodo}."
        )

# R7b
for (sala, dia, periodo), aulas in por_sala_slot.items():
    if len(aulas) > 1:
        erros["R7"].append(
            f"{sala}: {len(aulas)} aulas simultâneas em {dia}, período
{periodo}."
        )

resultados = {
    requisito: {"ok": not lista_erros, "erros": lista_erros}
}

```

```

        for requisito, lista_erros in erros.items()
    }
    return {
        "resultados": resultados,
        "valido_R1_R7": all(resultados[f"R{i}"]["ok"] for i in range(1, 8)),
        "R8_consistente_com_csv": resultados["R8"]["ok"],
    }

```

In []: *# [LLM] Contador independente de buracos segundo a definição de 01.*
Não consulta o modelo CP-SAT nem qualquer variável do solver.

```

def contar_buracos(horario):
    ocupacao_por_professor_dia = {}

    for aula in horario:
        chave = (
            aula["professor"],
            aula["dia"],
        )

        ocupacao_por_professor_dia.setdefault(
            chave,
            set(),
        ).add(aula["periodo"])

    total_buracos = 0
    detalhe = {}

    for chave, periodos_ocupados in sorted(
        ocupacao_por_professor_dia.items()
    ):
        primeiro_periodo = min(periodos_ocupados)
        ultimo_periodo = max(periodos_ocupados)

        periodos_buraco = [
            periodo
            for periodo in range(
                primeiro_periodo,
                ultimo_periodo + 1,
            )
            if periodo not in periodos_ocupados
        ]

        detalhe[chave] = {
            "periodos_ocupados": sorted(periodos_ocupados),
            "primeira_aula": primeiro_periodo,
            "ultima_aula": ultimo_periodo,
            "buracos": periodos_buraco,
            "numero_buracos": len(periodos_buraco),
        }

        total_buracos += len(periodos_buraco)

    return total_buracos, detalhe

```

```
In [ ]: # [LLM] Apresentação concisa dos resultados do validador.
def mostrar_validacao(resultado):
    for requisito, detalhe in resultado["resultados"].items():
        estado = "OK" if detalhe["ok"] else "FALHOU"
        sufixo = " (consistência com os CSV)" if requisito == "R8" else ""
        print(f"{requisito}: {estado}{sufixo}")
        for erro in detalhe["erros"]:
            print("  -", erro)
    print("R1-R7:", "VÁLIDO" if resultado["valido_R1_R7"] else "INVÁLIDO")
    print(
        "R8 - consistência com CSV:",
        "OK" if resultado["R8_consistente_com_csv"] else "FALHOU",
    )
```

```
In [ ]: # [LLM] Execução do validador independente sobre o horário base.
resultado_validacao = validar_horario(
    "dados",
    horario,
)

mostrar_validacao(resultado_validacao)
```

```
R1: OK
R2: OK
R3: OK
R4: OK
R5: OK
R6: OK
R7: OK
R8: OK (consistência com os CSV)
R1-R7: VÁLIDO
R8 - consistência com CSV: OK
```

Testes negativos do validador

Além da validação positiva, são criados horários deliberadamente inválidos para confirmar que o validador deteta violações de R1–R8. Estes testes são intencionalmente simples: não tentam isolar sempre uma única restrição. O critério de sucesso é o requisito alvo aparecer como violado; eventuais violações colaterais são mostradas no output.

```
In [ ]: # [LLM] Bateria simplificada de testes negativos de R1-R8.
# Parte do horário válido e introduz mutações claras; não tenta construir
# uma framework genérica de contraexemplos isolados.

def copia(h):
    return [dict(aula) for aula in h]

def duplicar_primeira(h):
    novo = copia(h)
    novo.append(dict(novo[0]))
    return novo

casos = {}
```

```

# R1, R3, R5 e R7: duplicar uma aula cria conflitos detetáveis nesses requisitos.
for requisito in ("R1", "R3", "R5", "R7"):
    casos[requisito] = duplicar_primeira(horario)

# R2: retirar um tempo letivo reduz a carga semanal observada.
casos["R2"] = copia(horario)[1:]

# R4: retirar um dos tempos da primeira disciplina dupla encontrada.
disciplinas_duplas = {
    d["disciplina"] for d in dados["disciplinas"] if d["duplo_periodo"] == "sim"
}
r4 = copia(horario)
indice_duplo = next(
    (i for i, aula in enumerate(r4) if aula["disciplina"] in disciplinas_duplas),
    None,
)
if indice_duplo is None:
    raise RuntimeError("Não existe disciplina dupla para testar R4.")
del r4[indice_duplo]
casos["R4"] = r4

# R6: mover uma aula de um professor para uma indisponibilidade real dos CSV.
indisponibilidade = next(iter(sorted(dados["indisponibilidades"])), None)
if indisponibilidade is None:
    raise RuntimeError("Não existe indisponibilidade para testar R6.")
professor, dia, periodo = indisponibilidade
r6 = copia(horario)
indice_professor = next(
    (i for i, aula in enumerate(r6) if aula["professor"] == professor),
    None,
)
if indice_professor is None:
    raise RuntimeError("O professor indisponível não tem aula no horário.")
r6[indice_professor]["dia"] = dia
r6[indice_professor]["periodo"] = periodo
casos["R6"] = r6

# R8: tornar o professor de uma entrada inconsistente com o CSV.
r8 = copia(horario)
r8[0]["professor"] = "__PROFESSOR_INCONSISTENTE__"
casos["R8"] = r8

resumo_testes_negativos = []
print("TESTES NEGATIVOS")
print("-" * 60)

for requisito, horario_invalido in casos.items():
    resultado = validar_horario("dados", horario_invalido)
    falhas = [
        r for r, detalhe in resultado["resultados"].items() if not detalhe["ok"]
    ]
    detetado = requisito in falhas
    resumo_testes_negativos.append(
        {"requisito": requisito, "detetado": detetado, "falhas": falhas}
    )

```

```

print(f"{requisito}: {'OK' if detetado else 'FALHOU'} | falhas: {falhas}")
assert detetado, f"O teste negativo de {requisito} não foi detetado."

print("Todos os requisitos alvo foram detetados: True")

```

TESTES NEGATIVOS

```

-----
R1: OK | falhas: ['R1', 'R2', 'R3', 'R5', 'R7']
R3: OK | falhas: ['R1', 'R2', 'R3', 'R5', 'R7']
R5: OK | falhas: ['R1', 'R2', 'R3', 'R5', 'R7']
R7: OK | falhas: ['R1', 'R2', 'R3', 'R5', 'R7']
R2: OK | falhas: ['R2']
R4: OK | falhas: ['R2', 'R4']
R6: OK | falhas: ['R4', 'R6']
R8: OK | falhas: ['R8']
Todos os requisitos alvo foram detetados: True

```

Execução de O1

Reconstrói-se o mesmo problema R1–R8 e acrescenta-se apenas O1. O horário otimizado é novamente verificado pelo validador independente. A célula de comparação apresenta, a partir da execução corrente, os buracos do horário sem objetivo, os do horário com O1, a soma das variáveis `buraco` e o valor da função objetivo.

```

In [ ]: # [LLM] Construção e resolução do mesmo problema R1-R8,
# acrescentando apenas O1.

modelo_o1, x_o1, y_o1, b_o1 = construir_modelo(
    dados
)

variaveis_o1 = adicionar_objetivo_buracos(
    modelo_o1,
    dados,
    x_o1,
)

solver_o1, estado_o1 = resolver_modelo(
    modelo_o1
)

print(
    "Estado com O1:",
    solver_o1.StatusName(estado_o1),
)

horario_o1 = extrair_horario(
    dados,
    x_o1,
    solver_o1,
    estado_o1,
)

if horario_o1 is None:

```

```

raise RuntimeError(
    "O modelo com 01 não produziu um horário."
)

```

Estado com 01: OPTIMAL

```

In [ ]: # [LLM] Validação independente do horário obtido com 01.
        resultado_validacao_o1 = validar_horario(
            "dados",
            horario_o1,
        )

        mostrar_validacao(
            resultado_validacao_o1
        )

```

R1: OK
R2: OK
R3: OK
R4: OK
R5: OK
R6: OK
R7: OK
R8: OK (consistência com os CSV)
R1-R7: VÁLIDO
R8 - consistência com CSV: OK

```

In [ ]: # [LLM] Comparação entre o contador independente, as variáveis de 01
        # e o valor da função objetivo reportado pelo solver.
        # Contagem do horário anteriormente obtido sem 01.
        buracos_sem_objetivo, detalhe_sem_objetivo = contar_buracos(
            horario
        )

        # Contagem independente do novo horário otimizado.
        buracos_com_o1, detalhe_com_o1 = contar_buracos(
            horario_o1
        )

        # Soma das próprias variáveis "buraco" do modelo.
        buracos_modelo = sum(
            solver_o1.Value(variavel)
            for variavel in variaveis_o1["buraco"].values()
        )

        # Valor da função objetivo reportado pelo CP-SAT.
        valor_objetivo = int(
            round(solver_o1.ObjectiveValue())
        )

        print("COMPARAÇÃO DE 01")
        print("-" * 50)

        print(
            "Buracos sem objetivo:",
            buracos_sem_objetivo,
        )

```

```

print(
    "Buracos com 01 - contador independente:",
    buracos_com_o1,
)

print(
    "Buracos com 01 - variáveis do modelo:",
    buracos_modelo,
)

print(
    "Função objetivo reportada pelo solver:",
    valor_objetivo,
)

print(
    "Redução observada:",
    buracos_sem_objetivo - buracos_com_o1,
)

assert (
    buracos_com_o1
    == buracos_modelo
    == valor_objetivo
), (
    "O contador independente e a implementação "
    "de 01 não produzem o mesmo valor."
)

```

COMPARAÇÃO DE 01

```

-----
Buracos sem objetivo: 14
Buracos com 01 - contador independente: 0
Buracos com 01 - variáveis do modelo: 0
Função objetivo reportada pelo solver: 0
Redução observada: 14

```

R9 — construção incremental

R9 parte de um horário existente **H0** e procura um novo horário **H1** quando os recursos mudam. Nesta implementação, `H0 = horario_o1`.

D6 — métrica de alteração. Uma alocação antiga é um tempo letivo completo

(turma, disciplina, dia, periodo, sala).

Uma alocação de H0 é preservada se a mesma combinação existir em H1. Aulas novas que não existiam em H0 não contam como aulas antigas alteradas. Consequentemente, mover totalmente um bloco duplo conta como duas alterações.

Se `A0` é o conjunto de alocações de H0, o número de alterações é

$$M(H0, H1) = |A0| - \sum_{a \in A0} x_a^{H1}.$$

D7 — estratégia incremental. O modelo de `dados_v2` minimiza explicitamente $M(H0, H1)$ e recebe $H0$ também como *hint*. O *hint* é apenas um ponto de partida sugerido ao solver; não é uma restrição nem substitui a função objetivo.

D8 — relação com O1. O1 não participa no objetivo de H1. Em R9 a prioridade desta experiência é preservar o horário anterior, tal como permitido pelo enunciado.

O mecanismo incremental não depende apenas do cenário `dados_v2`: uma nova indisponibilidade de professor pode ser acrescentada a `disponibilidade_excecoes.csv`, um aumento de disponibilidade corresponde a remover uma exceção, uma nova turma pode ser acrescentada a `turmas.csv` e a substituição de um professor pode ser feita em `disciplinas.csv`. Uma redução permanente da capacidade de salas pode ser representada diminuindo a respetiva `quantidade` em `salas.csv`; o formato atual, porém, não permite indicar que uma sala fica indisponível apenas em determinados períodos.

Para comparação foi construída uma **baseline** que resolve `dados_v2` desde zero apenas com R1–R8, sem utilizar $H0$ durante a resolução. As duas abordagens usam a mesma semente e uma única *worker* para tornar a medição mais reproduzível.

In []: *# [LLM] Infraestrutura comum de R9: métrica D6, objetivo, hint e resolução.
A mesma função incremental é reutilizada em dados_v2 e dados_reparo.*

```
from time import perf_counter

SEMENTE_R9 = 0
NUM_WORKERS_R9 = 1
LIMITE_TEMPO_R9 = None

def conjunto_alocacoes(horario):
    return {
        (a["turma"], a["disciplina"], a["dia"], a["periodo"], a["sala"])
        for a in horario
    }

def contar_alteracoes(h0, h1):
    a0, a1 = conjunto_alocacoes(h0), conjunto_alocacoes(h1)
    preservadas = a0 & a1
    alteradas = a0 - a1
    return {
        "total_h0": len(a0),
        "preservadas": len(preservadas),
        "alteradas": len(alteradas),
        "novas": len(a1 - a0),
    }

def criar_indices_dados(dados):
    return {
```



```

        "turmas": {v: i for i, v in enumerate(dados["turmas"])},
        "disciplinas": {
            v["disciplina"]: i for i, v in enumerate(dados["disciplinas"])
        },
        "dias": {v: i for i, v in enumerate(dados["dias"])},
        "periodos": {v: i for i, v in enumerate(dados["periodos"])},
        "salas": {v["id"]: i for i, v in enumerate(dados["salas"])},
    }

def chave_x(allocacao, indices):
    valores = (
        indices["turmas"].get(allocacao[0]),
        indices["disciplinas"].get(allocacao[1]),
        indices["dias"].get(allocacao[2]),
        indices["periodos"].get(allocacao[3]),
        indices["salas"].get(allocacao[4]),
    )
    return None if None in valores else valores

def adicionar_objetivo_e_hint(modelo, dados_novos, x, h0):
    indices = criar_indices_dados(dados_novos)
    variaveis_preservadas = []
    alteracoes_forçadas = 0
    hints_adicionados = 0
    hints_ignorados = 0

    for allocacao in conjunto_allocacoes(h0):
        chave = chave_x(allocacao, indices)
        if chave is None or chave not in x:
            alteracoes_forçadas += 1
            hints_ignorados += 1
            continue
        variavel = x[chave]
        variaveis_preservadas.append(variavel)
        modelo.AddHint(variavel, 1)
        hints_adicionados += 1

    modelo.Minimize(
        alteracoes_forçadas + sum(1 - v for v in variaveis_preservadas)
    )
    return {
        "hints_adicionados": hints_adicionados,
        "hints_ignorados": hints_ignorados,
        "alteracoes_forçadas": alteracoes_forçadas,
    }

def resolver_medido(modelo):
    solver = cp_model.CpSolver()
    solver.parameters.random_seed = SEMENTE_R9
    solver.parameters.num_search_workers = NUM_WORKERS_R9
    if LIMITE_TEMPO_R9 is not None:
        solver.parameters.max_time_in_seconds = LIMITE_TEMPO_R9

    inicio = perf_counter()
    estado = solver.Solve(modelo)
    tempo = perf_counter() - inicio

```

```

    return solver, estado, {
        "tempo_externo_s": tempo,
        "wall_time_solver_s": solver.WallTime(),
    }

def resolver_incremental(dados_novos, h0):
    modelo, x, _, _ = construir_modelo(dados_novos)
    info_hint = adicionar_objetivo_e_hint(modelo, dados_novos, x, h0)
    solver, estado, medicao = resolver_medido(modelo)
    horario_novo = extrair_horario(dados_novos, x, solver, estado)
    if horario_novo is None:
        raise RuntimeError("A resolução incremental não produziu um horário.")
    return {
        "solver": solver,
        "estado": estado,
        "medicao": medicao,
        "horario": horario_novo,
        "hint": info_hint,
    }

def resolver_desde_zero(dados_novos):
    modelo, x, _, _ = construir_modelo(dados_novos)
    solver, estado, medicao = resolver_medido(modelo)
    horario_novo = extrair_horario(dados_novos, x, solver, estado)
    if horario_novo is None:
        raise RuntimeError("A resolução desde zero não produziu um horário.")
    return {
        "solver": solver,
        "estado": estado,
        "medicao": medicao,
        "horario": horario_novo,
    }

```

```

In [ ]: # [LLM] Experiência principal de R9: dados -> dados_v2.
        # A baseline não usa H0 durante a resolução; H0 só é usado depois para medir
        # alterações.

        H0 = horario_o1
        dados_v2 = carregar_dados("dados_v2")

        resultado_incremental = resolver_incremental(dados_v2, H0)
        validacao_incremental = validar_horario(
            "dados_v2", resultado_incremental["horario"]
        )
        alteracoes_incremental = contar_alteracoes(
            H0, resultado_incremental["horario"]
        )
        objetivo_incremental =
        int(round(resultado_incremental["solver"].ObjectiveValue()))
        assert alteracoes_incremental["alteradas"] == objetivo_incremental

        resultado_zero = resolver_desde_zero(dados_v2)
        validacao_zero = validar_horario("dados_v2", resultado_zero["horario"])
        alteracoes_zero = contar_alteracoes(H0, resultado_zero["horario"])

        print("R9 - CENÁRIO OFICIAL dados -> dados_v2")

```

```

print("-" * 60)
print(
    "Incremental | estado:",
    resultado_incremental["solver"].StatusName(resultado_incremental["estado"]),
)
print("Incremental | tempo:", resultado_incremental["medicao"]["tempo_externo_s"])
print("Incremental | WallTime:", resultado_incremental["medicao"]
["wall_time_solver_s"])
print("Incremental | hints:", resultado_incremental["hint"])
print("Incremental | preservadas:", alteracoes_incremental["preservadas"])
print("Incremental | alteradas:", alteracoes_incremental["alteradas"])
print("Incremental | objetivo:", objetivo_incremental)
print("Incremental | R1-R7:", "VÁLIDO" if validacao_incremental["valido_R1_R7"]
else "INVÁLIDO")
print("Incremental | R8 CSV:", "OK" if
validacao_incremental["R8_consistente_com_csv"] else "FALHOU")
print()
print(
    "Desde zero | estado:",
    resultado_zero["solver"].StatusName(resultado_zero["estado"]),
)
print("Desde zero | tempo:", resultado_zero["medicao"]["tempo_externo_s"])
print("Desde zero | WallTime:", resultado_zero["medicao"]["wall_time_solver_s"])
print("Desde zero | preservadas:", alteracoes_zero["preservadas"])
print("Desde zero | alteradas:", alteracoes_zero["alteradas"])
print("Desde zero | R1-R7:", "VÁLIDO" if validacao_zero["valido_R1_R7"] else
"INVÁLIDO")
print("Desde zero | R8 CSV:", "OK" if validacao_zero["R8_consistente_com_csv"]
else "FALHOU")
print()
print(
    "Diferença de tempo (zero - incremental):",
    resultado_zero["medicao"]["tempo_externo_s"]
    - resultado_incremental["medicao"]["tempo_externo_s"],
)
print(
    "Alterações evitadas pelo incremental:",
    alteracoes_zero["alteradas"] - alteracoes_incremental["alteradas"],
)

assert validacao_incremental["valido_R1_R7"]
assert validacao_incremental["R8_consistente_com_csv"]
assert validacao_zero["valido_R1_R7"]
assert validacao_zero["R8_consistente_com_csv"]

resumo_r9 = {
    "incremental": {
        "estado":
resultado_incremental["solver"].StatusName(resultado_incremental["estado"]),
        "tempo_externo_s": resultado_incremental["medicao"]["tempo_externo_s"],
        "alteracoes": alteracoes_incremental["alteradas"],
        "preservadas": alteracoes_incremental["preservadas"],
        "objetivo": objetivo_incremental,
    },
    "desde_zero": {
        "estado": resultado_zero["solver"].StatusName(resultado_zero["estado"]),

```

```

    "tempo_externo_s": resultado_zero["medicao"]["tempo_externo_s"],
    "alteracoes": alteracoes_zero["alteradas"],
    "preservadas": alteracoes_zero["preservadas"],
  },
}

```

R9 - CENÁRIO OFICIAL dados -> dados_v2

```

-----
Incremental | estado: OPTIMAL
Incremental | tempo: 0.07740039999771398
Incremental | WallTime: 0.076908
Incremental | hints: {'hints_adicionados': 34, 'hints_ignorados': 0, 'alteracoes_for
cadas': 0}
Incremental | preservadas: 32
Incremental | alteradas: 2
Incremental | objetivo: 2
Incremental | R1-R7: VÁLIDO
Incremental | R8 CSV: OK

```

```

Desde zero | estado: OPTIMAL
Desde zero | tempo: 0.1082679999995274
Desde zero | WallTime: 0.1077284
Desde zero | preservadas: 3
Desde zero | alteradas: 31
Desde zero | R1-R7: VÁLIDO
Desde zero | R8 CSV: OK

```

Diferença de tempo (zero - incremental): 0.03086760000223876

Alterações evitadas pelo incremental: 29

Interpretação do cenário principal

A célula anterior mostra os resultados efetivamente medidos para `dados → dados_v2`. A comparação relevante é feita em duas dimensões: tempo de resolução e número de alocações antigas preservadas. Como a instância é pequena e a medição temporal corresponde a uma execução, não se generaliza o desempenho a partir destes tempos. Os valores apresentados no output são a única fonte dos resultados experimentais.

Cenário controlado de reparação

O cenário oficial pode não obrigar H0 a mudar. Para exercitar uma reparação real e, simultaneamente, testar R8 com outro conjunto de CSV, o notebook cria `dados_reparo/` a partir de `dados_v2/` e de uma aula real de H0.

As aulas são percorridas numa ordem determinística, dando prioridade a disciplinas sem bloco duplo. Para cada candidata acrescenta-se uma indisponibilidade do respetivo professor exatamente nesse dia e período. Se a instância ficar inviável, passa-se à candidata seguinte. É usada a **primeira candidata reparável**, não a que produz menos alterações. Depois aplica-se exatamente a mesma função incremental usada em `dados_v2`.

In []: *# [LLM] Criação genérica e reproduzível do conjunto dados_reparo.*

```
def criar_cenario_reparo(
    h0,
    dados_base,
    pasta_base="dados_v2",
    pasta_destino="dados_reparo",
    semente=0,
    num_workers=1,
):
    import shutil

    disciplinas = {d["disciplina"]: d for d in dados_base["disciplinas"]}
    ordem_dias = {dia: i for i, dia in enumerate(dados_base["dias"])}

    candidatos = sorted(
        (dict(aula) for aula in h0 if aula["disciplina"] in disciplinas),
        key=lambda aula: (
            disciplinas[aula["disciplina"]]["duplo_periodo"] == "sim",
            aula["turma"],
            ordem_dias.get(aula["dia"], len(ordem_dias)),
            aula["periodo"],
            aula["disciplina"],
            aula["sala"],
        ),
    )

    escolhida = None
    execucao = None
    candidatos_testados = 0

    for aula in candidatos:
        professor = disciplinas[aula["disciplina"]]["professor"]
        if aula["professor"] != professor:
            continue
        nova_excecao = (professor, aula["dia"], aula["periodo"])
        if nova_excecao in dados_base["indisponibilidades"]:
            continue

        candidatos_testados += 1
        dados_teste = dict(dados_base)
        dados_teste["indisponibilidades"] = (
            set(dados_base["indisponibilidades"]) | {nova_excecao}
        )
        modelo_teste, _, _, _ = construir_modelo(dados_teste)
        solver_teste = cp_model.CpSolver()
        solver_teste.parameters.random_seed = semente
        solver_teste.parameters.num_search_workers = num_workers
        estado_teste = solver_teste.Solve(modelo_teste)

        if estado_teste in (cp_model.OPTIMAL, cp_model.FEASIBLE):
            escolhida, execucao = aula, nova_excecao
            break

    if escolhida is None:
```

```

        raise RuntimeError(
            "Não foi encontrada uma indisponibilidade derivada de H0 que "
            "mantenha a instância satisfazível."
        )

    origem, destino = Path(pasta_base), Path(pasta_destino)
    if destino.exists():
        shutil.rmtree(destino)
    shutil.copytree(origem, destino)

    caminho = destino / "disponibilidade_excecoes.csv"
    with open(caminho, newline="", encoding="utf-8-sig") as f:
        leitor = csv.DictReader(f)
        fieldnames = leitor.fieldnames
        linhas = list(leitor)
    if fieldnames is None:
        raise ValueError("disponibilidade_excecoes.csv não tem cabeçalho.")

    linhas.append(
        {"professor": excecao[0], "dia": excecao[1], "periodo": str(excecao[2])}
    )
    with open(caminho, "w", newline="", encoding="utf-8-sig") as f:
        escritor = csv.DictWriter(f, fieldnames=fieldnames)
        escritor.writeheader()
        escritor.writerows(linhas)

    return {
        "aula_afetada": escolhida,
        "nova_indisponibilidade": {
            "professor": excecao[0], "dia": excecao[1], "periodo": excecao[2]
        },
        "candidatos_testados": candidatos_testados,
        "pasta": str(destino),
    }

```

In []: # [LLM] Execução do cenário de reparação e teste com CSV alternativos.

```

cenario_reparo = criar_cenario_reparo(
    H0,
    dados_v2,
    semente=SEMENTE_R9,
    num_workers=NUM_WORKERS_R9,
)
dados_reparo = carregar_dados("dados_reparo")

validacao_h0 = validar_horario("dados_reparo", H0)
outros_requisitos = ("R1", "R2", "R3", "R4", "R5", "R7", "R8")
outros_ok = all(validacao_h0["resultados"][r]["ok"] for r in outros_requisitos)
assert not validacao_h0["resultados"]["R6"]["ok"]
assert outros_ok

resultado_reparo = resolver_incremental(dados_reparo, H0)
horario_reparo = resultado_reparo["horario"]
validacao_reparo = validar_horario("dados_reparo", horario_reparo)
alteracoes_reparo = contar_alteracoes(H0, horario_reparo)
objetivo_reparo = int(round(resultado_reparo["solver"].ObjectiveValue()))

```

```

print("CENÁRIO CONTROLADO DE REPARAÇÃO")
print("-" * 60)
print("Aula de H0 afetada:", cenario_reparo["aula_afetada"])
print("Nova indisponibilidade:", cenario_reparo["nova_indisponibilidade"])
print("Candidatos testados:", cenario_reparo["candidatos_testados"])
print("H0 perante dados_reparo | R6: FALHOU")
print("Restantes R1-R5, R7 e R8:", "OK" if outros_ok else "FALHOU")
print()
print(
    "H1 reparado | estado:",
    resultado_reparo["solver"].StatusName(resultado_reparo["estado"]),
)
print("H1 reparado | tempo:", resultado_reparo["medicao"]["tempo_externo_s"])
print("H1 reparado | hints:", resultado_reparo["hint"])
print("H1 reparado | preservadas:", alteracoes_reparo["preservadas"])
print("H1 reparado | alteradas:", alteracoes_reparo["alteradas"])
print("H1 reparado | objetivo:", objetivo_reparo)
print("H1 reparado | R1-R7:", "VÁLIDO" if validacao_reparo["valido_R1_R7"] else "INVÁLIDO")
print("H1 reparado | R8 CSV alternativo:", "OK" if
validacao_reparo["R8_consistente_com_csv"] else "FALHOU")

assert validacao_reparo["valido_R1_R7"]
assert validacao_reparo["R8_consistente_com_csv"]
assert alteracoes_reparo["alteradas"] >= 1
assert alteracoes_reparo["alteradas"] == objetivo_reparo

```

CENÁRIO CONTROLADO DE REPARAÇÃO

```

-----
Aula de H0 afetada: {'turma': '7ºA', 'disciplina': 'Matemática', 'professor': 'Prof.
Ana', 'dia': 'Seg', 'periodo': 1, 'sala': 'Sala Normal #2'}
Nova indisponibilidade: {'professor': 'Prof. Ana', 'dia': 'Seg', 'periodo': 1}
Candidatos testados: 1
H0 perante dados_reparo | R6: FALHOU
Restantes R1-R5, R7 e R8: OK

H1 reparado | estado: OPTIMAL
H1 reparado | tempo: 0.0751414999976987
H1 reparado | hints: {'hints_adicionados': 34, 'hints_ignorados': 0, 'alteracoes_for
cadas': 0}
H1 reparado | preservadas: 31
H1 reparado | alteradas: 3
H1 reparado | objetivo: 3
H1 reparado | R1-R7: VÁLIDO
H1 reparado | R8 CSV alternativo: OK

```

Matriz de rastreabilidade final

Item	Origem	Decisão de modelação / implementação	Contribuição da LLM	Evidência no notebook
R1	Enunciado	soma das alocações da turma por slot <code><= 1</code>	formalização e código	validador positivo + teste negativo

Item	Origem	Decisão de modelação / implementação	Contribuição da LLM	Evidência no notebook
R2	Enunciado	carga semanal representada por igualdade exata	formalização e código	validador positivo + teste negativo
R3	Enunciado	normal: máx. 1 período/dia; dupla: máx. 1 bloco/dia	formalização e código	validador positivo + teste negativo
R4	Enunciado	início de bloco <code>b</code> e dois períodos consecutivos; D3 não exige mesma sala	alternativas, formalização e código	validador positivo + teste negativo
R5	Enunciado	máx. 1 aula/professor/slot	formalização e código	validador positivo + teste negativo
R6	Enunciado	ocupação proibida nas exceções dos CSV	formalização e código	validador, teste negativo e <code>dados_reparo</code>
R7	Enunciado	D1 expande quantidades em salas concretas; compatibilidade + exclusividade	proposta, discussão e código	validador positivo + teste negativo
R8	Enunciado	leitura parametrizada dos CSV; D4 rejeita carga ímpar numa disciplina dupla	carregador, validador e testes	consistência nos CSV + conjunto alternativo <code>dados_reparo</code>
O1	Enunciado	D5: <code>ocupado</code> , <code>tem_antes</code> , <code>tem_depois</code> , <code>buraco</code>	formalização, equivalências e código	contador independente comparado com o objetivo
R9	Enunciado	H0= <code>horario_o1</code> ; D6 alteração por tempo letivo; D7 objetivo + hint; D8 O1 fora do objetivo	alternativas, formalização e código	cenário oficial + cenário de reparação

Limitações

- As salas equivalentes recebem identificadores artificiais (`#1` , `#2` , ...), introduzindo simetria entre salas do mesmo tipo.
- A métrica D6 conta alterações por tempo letivo; deslocar um bloco duplo completo conta como duas alterações.
- Os tempos de R9 são medidos numa instância pequena e numa execução, pelo que não sustentam conclusões gerais de desempenho.
- O formato atual dos CSV não representa a indisponibilidade de uma sala apenas em períodos específicos.
- Não foi realizada a experiência de escala apresentada como extensão opcional no enunciado específico.

Reprodutibilidade

São necessários `horario_escolar.py` e as pastas `dados/` e `dados_v2/`. A pasta `dados_reparo/` é recriada automaticamente durante a execução. O cabeçalho declara `marimo` e `ortools`; as restantes bibliotecas usadas pertencem à biblioteca standard de Python.

O trabalho foi desenvolvido e executado num ambiente Conda dedicado (`logica`) com:

- Python 3.14.7
- Marimo 0.25.0
- OR-Tools 9.15.6755

O cabeçalho do notebook declara `requires-python = ">=3.14"` e as dependências externas `marimo` e `ortools`.

Para a experiência R9 usa-se semente fixa, uma única *worker* e nenhum limite explícito de tempo.

Utilização de LLM

O código Python deste notebook foi **maioritariamente gerado com auxílio de uma LLM**. As componentes geradas estão identificadas por comentários `[LLM]`. A LLM apoiou a interpretação dos requisitos, a formalização, a implementação, os testes e a organização do relatório. As decisões de modelação foram discutidas e revistas pelo grupo, e os resultados apresentados no notebook são produzidos pelas execuções efetuadas pelo grupo.

Ligação para o diálogo LLM usado no desenvolvimento: [Diálogo completo com a LLM](#)

Conclusão

A solução cobre R1–R8, O1 e R9, com validação independente e um conjunto alternativo de CSV. Os resultados quantitativos não são duplicados neste texto: ficam nos outputs das células que os calculam, evitando divergências entre o relatório e a execução final.